

a rectangle in 2D. Before moving to the next steps, it is imperative that you understand the theoretical basis for using these inbuilt functions.

Right from the beginning, we have used some inbuilt functions that you might be able to guess the meaning of. First, we have defined *fov*, which is the field of view that we will have while viewing the game. Following that, we define the camera of the Z axis, which is basically the parameter we use to control the clipping planes in the Z axis.

Note: Clipping planes are imaginary planes that are used to limit the region in which an object will be rendered.

Then comes one of the most important aspects of the code — the perspective, which decides how the objects will get rendered and be shown on screen. The parameters here are field of view, aspect ratio, near clip and far clip.

Note: In case this is your maiden 3D project using Processing, or if you are new to 3D programming, in general, I urge you to check out the Processing docs for more detailed information, starting with the P3D tutorial by Daniel Shiffman at <https://processing.org/tutorials/p3d/>.

We can now move on to the next part of drawing the path, which is creating the cuboid in the appropriate location. As you can see, the next segment of the code is enclosed between a push-pop matrix stack.

Here's another short lesson on the theoretical aspects — a matrix stack is a technique that is used a lot by anyone working on graphics. It allows the programmer to achieve a stack of consecutive code that will not affect anything outside of this matrix stack. So, putting it simply, any code written between the push-pop stack will not be reflected outside the stack segment.

Moving back to our code, the first line we have in our stack is the translation of the axis to the right place. Since by default the axis origin is present at the top left corner of the

screen, to make matters easy, we will translate the axis to where we require the object, and then draw it. We then decide the colour of the cuboid, and draw the actual cuboid. We achieve this by calling *box()*, which takes in three arguments — the lengths of the box in the X, Y and Z axes.

Once we have drawn what we want, we call *popMatrix()* to revert any changes made. This resets changes to the axes such as translation or rotation, leaving us again with a clean slate for any other transformations that we need later in the code.

Feel free to play around with the values everywhere to get a good feel of what you are changing and how that affects the output, as this will give you a good idea of how you can improve this game, or start any other project that you have in mind.

If you have been following the discussion, your output should be looking like what's shown in Figure 1.

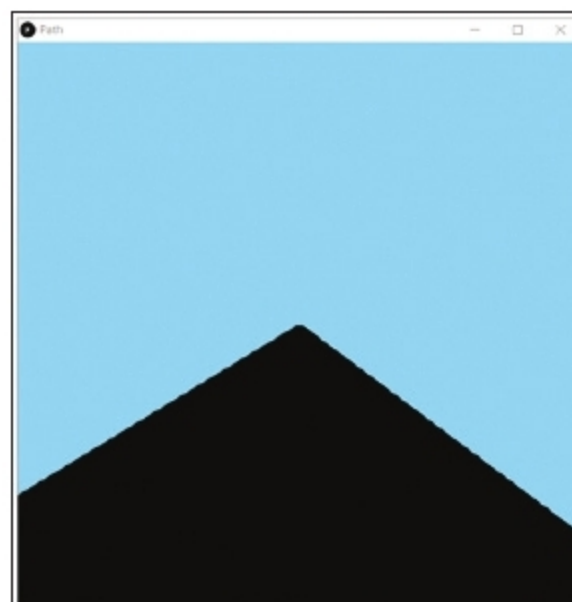


Figure 1: Basic path

Making the obstacles

The next part of our little game will be to implement some sort of obstacle for the player to jump over. Here, considering a first player view, we shall make our obstacles a wall-like structure, again made using *box()*.

But before moving on to drawing the wall, for ease of use, let us make an obstacle class with what will be related to the obstacle defined.

```
class obstacle:
```

```
def __init__(self, z, ht):
    self.z = z
    self.ht = ht
def move(self):
    self.z += 10
def check_bounds(self):
    if self.z > 100:
        self.z = -500*10
        self.ht = random(100, 150)
```

Now that we have the class *obstacle*, we can create a list of obstacles to make. We will do this in the *setup()*, just like in the previous game.

```
z_val = -500*10
for i in range(6):
    o = obstacle(z_val, random(100,
150))
    obs.append(o)
    z_val -= 850
```

As you can see, we start off with the obstacles that are far off on the game screen with a very high negative Z axis value. We also randomise the height so as to present some challenge for the game player. We then append the obstacle to the list and change the Z axis value for the next object.

With that, we can now draw our obstacle in *draw()*, using the following commands:

```
for ob in obs:
    pushMatrix()
    translate(width/2, base_ht-ob.
ht, ob.z)
    fill(255)
    box(200, ob.ht, 50)
    popMatrix()
    ob.move()
    ob.check_bounds()
```

Now for the explanation—as in the previous game, we iterate through all the objects in the list. For each object, we again use a push-pop matrix stack to change the axis to the correct location.

Here, the reason we need the height to be the base height minus the object height is because the box will be drawn starting from wherever the origin is. In order to avoid unnatural overlap, we